**Universidad Tecnológica del Sureste de Veracruz**
Ingeniería en Desarrollo y Gestión de Software

**Desarrollo Móvil Integral**
Grupo 1001



RESULTADO DE APRENDIZAJE · VERSIÓN RESUMIDA

# Flashly v0.4.0

## Documentación de la aplicación móvil: repositorio, metodología ágil, arquitectura, patrones, frameworks, pruebas, seguridad, APIs de terceros y servicios en la nube

EQUIPO

**José Jamin García García**
**Jorge Israel Aguilera Rueda**
**Bárbara Alejandra Díaz Pérez**

DOCENTE

Mtro. Isaías Gallegos Salvador

FECHA

28 de octubre de 2026

Nanchital, Veracruz

github.com/Jamgargar04/flashly

# 01 Datos generales y cumplimiento

Flashly es una app móvil para la comunidad de la UTSV hecha con React Native, Expo y Firebase. Este resumen muestra la evidencia de cada punto del resultado de aprendizaje en la versión v0.4.0.

| Dato | Valor |
|---|---|
| **Materia** | Desarrollo Móvil Integral · Ingeniería en Desarrollo y Gestión de Software · UTSV |
| **Docente** | Mtro. Isaías Gallegos Salvador |
| **Equipo** | José Jamin García García, Jorge Israel Aguilera Rueda y Bárbara Alejandra Díaz Pérez · grupo 1001 |
| **Versión** | v0.4.0 · commit `fcc738c` · rama `develop` · 15 de octubre de 2026 |
| **Repositorio** | github.com/Jamgargar04/flashly |

> *Desarrolla una aplicación móvil utilizando un repositorio de código fuente, con base en un caso de estudio de acuerdo a la metodología ágil, la arquitectura de software, los patrones de diseño, frameworks y esquema de pruebas seleccionados, que incluya: mecanismos de seguridad, API's de terceros, servicios en la nube y enlace del repositorio en funcionamiento.*

| Requisito | Evidencia en Flashly | Sección |
|---|---|---|
| Repositorio de código fuente | GitHub público, 17 commits, ramas main y develop, CI en cada push | 03 |
| Caso de estudio | Poca convivencia entre estudiantes de la UTSV | 02 |
| Metodología ágil | Scrum adaptado: backlog, 5 incrementos, definición de terminado | 03 |
| Arquitectura de software | Clean Architecture en 4 capas | 04 |
| Patrones de diseño | Repository, Strategy, Adapter, Factory, Singleton, Observer, DI, Store | 04 |
| Frameworks | Expo SDK 57, React Native 0.86, Expo Router, Zustand, Zod, Firebase 12 | 05 |
| Esquema de pruebas | 56 pruebas con Jest, TypeScript estricto, GitHub Actions | 05 |
| Mecanismos de seguridad | Correo institucional verificado, reglas de Firestore y Storage, HTTPS | 06 |
| APIs de terceros | Cloudinary Upload API y Google Cloud Vision | 07 |
| Servicios en la nube | Firebase Authentication, Cloud Firestore, Cloud Functions, EAS | 07 |
| Enlace en funcionamiento | Repositorio con CI en verde y APK instalable | 08 y 09 |

## 02 Caso de estudio

El caso de estudio es la poca convivencia en la UTSV: se pasan años en el mismo campus y cada quien conoce solo a su grupo. Flashly propone un aviso sorpresa al día: todos tienen 2 minutos para compartir una foto con la cámara trasera y otra con la frontal, y para ver los momentos de los demás primero hay que publicar el propio.

**Reglas del negocio:**

1. La ventana cae entre las 9:00 y las 20:00, dura 2 minutos y es igual en todos los teléfonos (hash de la fecha).
2. Un momento por persona al día: el id siempre es `día_usuario`.
3. Si no publicas, no ves: el feed se desbloquea al compartir tu momento.
4. Publicar fuera de los 2 minutos se permite, pero queda marcado como "Tarde".
5. Los momentos se ocultan a las 24 horas y solo entran correos institucionales verificados.

## 03 Repositorio y metodología ágil

El código está en un repositorio público de GitHub (github.com/Jamgargar04/flashly) con 17 commits que siguen Conventional Commits (7 feat, 6 fix, 4 chore). `main` guarda la versión estable y `develop` integra cada incremento. GitHub Actions revisa tipos y corre las pruebas en cada push: lleva 14 ejecuciones y la #14, sobre la v0.4.0, pasó en 53 s.

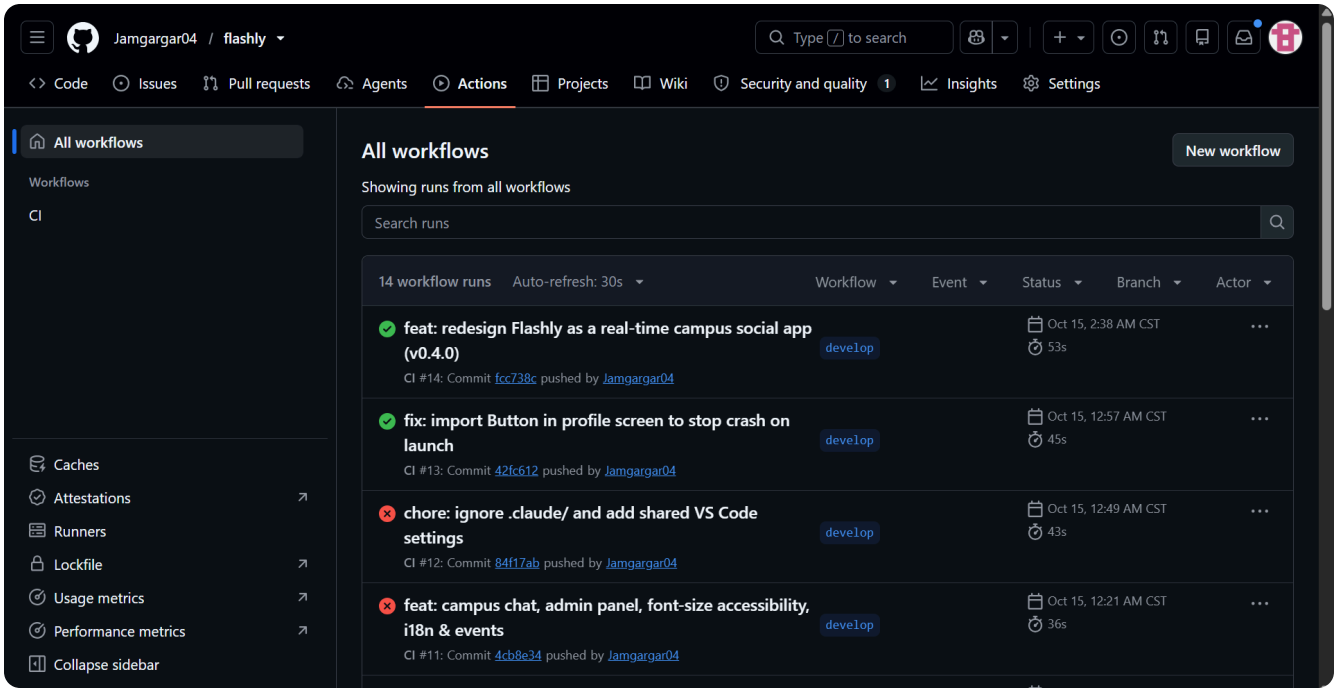



**Figura 1.** Ejecuciones de GitHub Actions; la #14 de la v0.4.0 en verde y las fallidas #11 y #12 corregidas en el commit siguiente.

**Metodología ágil.** Se trabajó con Scrum adaptado a un equipo de tres: un backlog de 14 historias de usuario (11 terminadas en la v0.4.0 y 3 pendientes para producción), entregas en incrementos funcionales y una definición de terminado en la plantilla de Pull Request: pasa tipos y pruebas, se probó en Android, no expone claves y queda documentado.

| Incremento | Fecha | Qué entregó |
|---|---|---|
| 5 · v0.4.0 | 15 oct 2026 | Rediseño como red social del campus en tiempo real |
| 4 · Funciones sociales | 14–15 oct | Maestros, tema oscuro, español e inglés, lugares, eventos, chat y panel de admin |
| 3 · Estabilización Android | 14 oct | Subida de fotos a Cloudinary desde Android |
| 2 · Conexión a la nube | 14 oct | Firebase, Cloudinary y proyecto de Expo |
| 1 · Proyecto inicial | 14 oct | Arquitectura, dominio, modo demostración, pruebas y CI |

## 04 Arquitectura y patrones de diseño

Flashly usa Clean Architecture en cuatro capas. Las reglas del negocio no dependen de Expo, Firebase ni Cloudinary, por eso la app funciona en modo demostración y en modo nube cambiando solo la configuración.

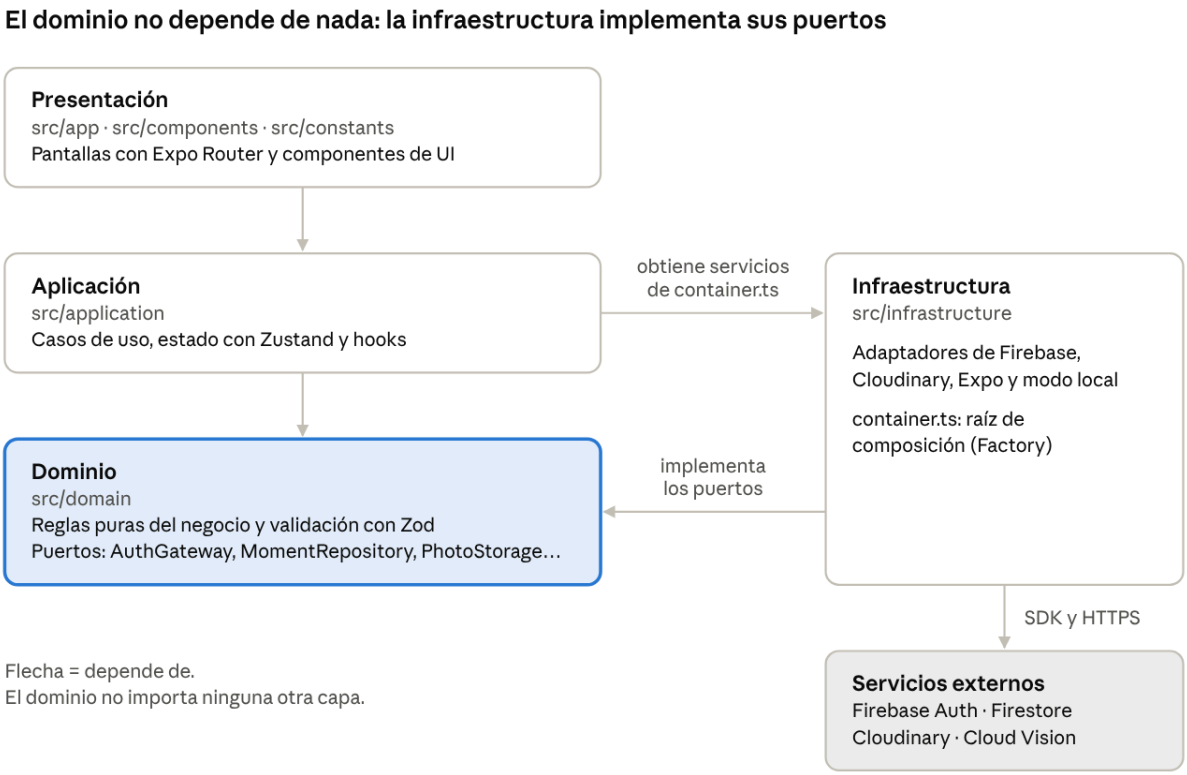



**Figura 2.** Capas y dependencias. Solo `container.ts` conoce las clases concretas; el resto depende de los puertos del dominio.

| Patrón | Dónde se aplica |
|---|---|
| Repository | `FirestoreMomentRepository` y `LocalMomentRepository` detrás del puerto `MomentRepository` |
| Strategy | `PhotoStorage`: Cloudinary, Firebase Storage o local, sin tocar el caso de uso |
| Adapter | `FirebaseAuthGateway`, `CloudinaryPhotoStorage`, `ExpoReminderScheduler` |
| Factory | `createServices()` arma las implementaciones según la configuración |
| Singleton | `getFirebase()` inicializa Firebase una sola vez |
| Observer | `onSnapshot` actualiza sesión, feed y chat en tiempo real |
| Inyección de dependencias | `createPublishMoment({ compressor, photos, moments })` se prueba sin servicios reales |
| Store (Flux) | `useSession`, `useFeed`, `useEvents` con Zustand |

Los principios SOLID se ven en esos mismos archivos: cada adaptador hace una sola cosa, un proveedor nuevo es una clase nueva, los repositorios local y de Firestore son intercambiables, los puertos son pequeños y los casos de uso dependen de interfaces, no de Firebase.

## 05 Frameworks y esquema de pruebas

| Framework o herramienta | Versión | Uso |
|---|---|---|
| Expo SDK · React Native · React | 57 · 0.86.3 · 19.2 | Base de la app móvil y compilación |
| TypeScript (estricto) | 6.0 | Tipado de todo el proyecto |
| Expo Router | 57 | Navegación por archivos |
| Zustand · Zod | 5 · 4 | Estado global y validación |
| Firebase JS SDK | 12 | Authentication y Firestore |
| Jest + jest-expo | 29 | Pruebas automáticas |

**Esquema de pruebas.** La v0.4.0 pasa 56 pruebas automáticas en 9 suites de Jest y compila sin errores con TypeScript estricto; GitHub Actions repite ambas revisiones en cada push. La lógica crítica supera el 97 % de cobertura (dominio 97.9 %, aplicación 100 %). Además hay un plan de 10 pruebas manuales en el teléfono, y las pruebas de reglas con el emulador quedan como siguiente paso.

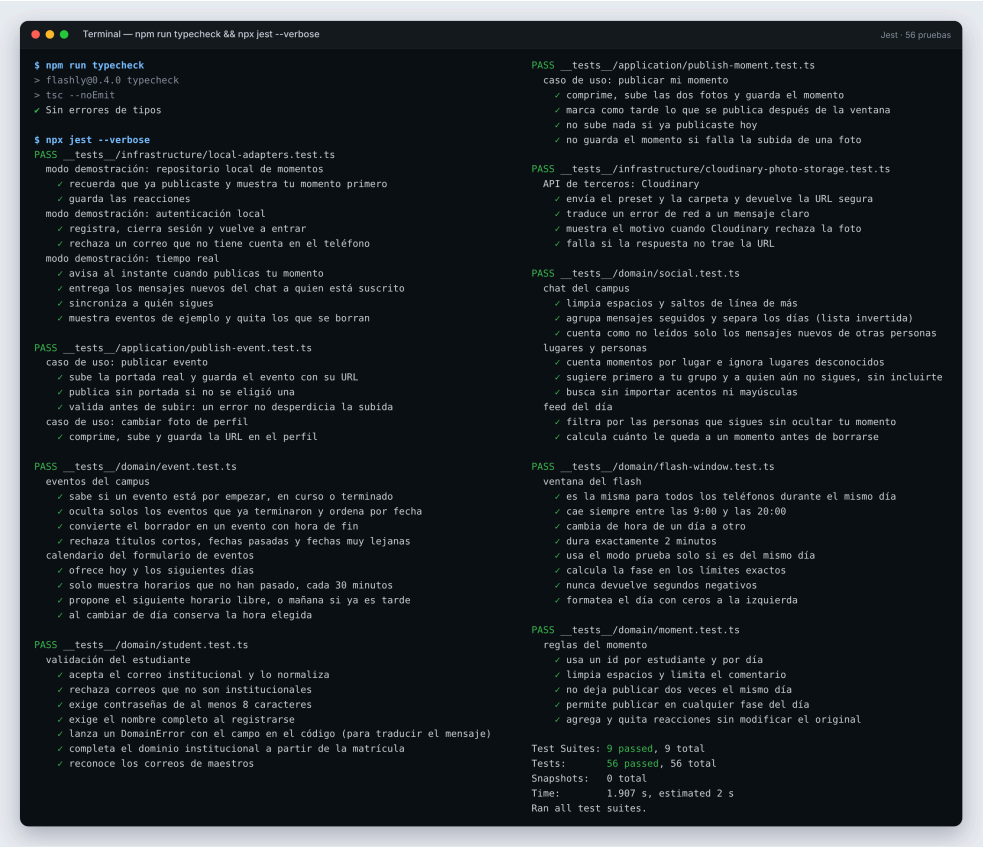



**Figura 3.** Revisión de tipos sin errores y las 56 pruebas en verde sobre la v0.4.0.

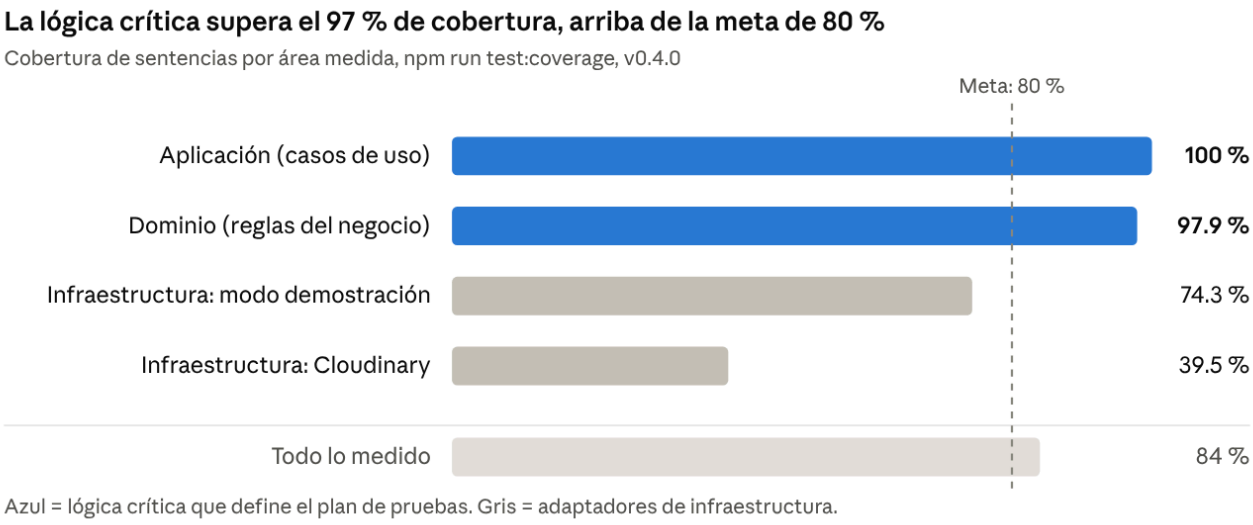



**Figura 4.** Cobertura de sentencias por área (npm run test:coverage).

## 06 Mecanismos de seguridad

La app valida para dar mensajes claros, y el servidor vuelve a validar todo con las reglas de Firestore y Storage, así un cliente modificado no puede saltárselas.

| Mecanismo | Cómo se aplica |
|---|---|
| Autenticación | Firebase Authentication con correo y contraseña de 8 caracteres o más |
| Solo comunidad UTSV | Correos `@alumnos.utsv.edu.mx` y `@utsv.edu.mx` verificados, revisados con Zod y en el servidor |
| Autorización | Cada quien escribe solo lo suyo, un momento por día y nadie se da permisos de admin |
| "Si no publicas, no ves" | Leer momentos ajenos exige tener el propio del día (regla del servidor) |
| Datos y archivos | Campos y longitudes limitados; solo URLs `https://`; fotos JPEG de menos de 5 MB |
| Secretos y permisos | `.env` y llaves de firma fuera del repo; Android solo pide la cámara |
| Retención y moderación | Momentos visibles 24 h, reportes anónimos y moderación por admins y Cloud Vision |




**Figura 5.** Reglas de Firestore: solo cuentas UTSV verificadas, id `día_usuario` y feed visible solo si ya publicaste.

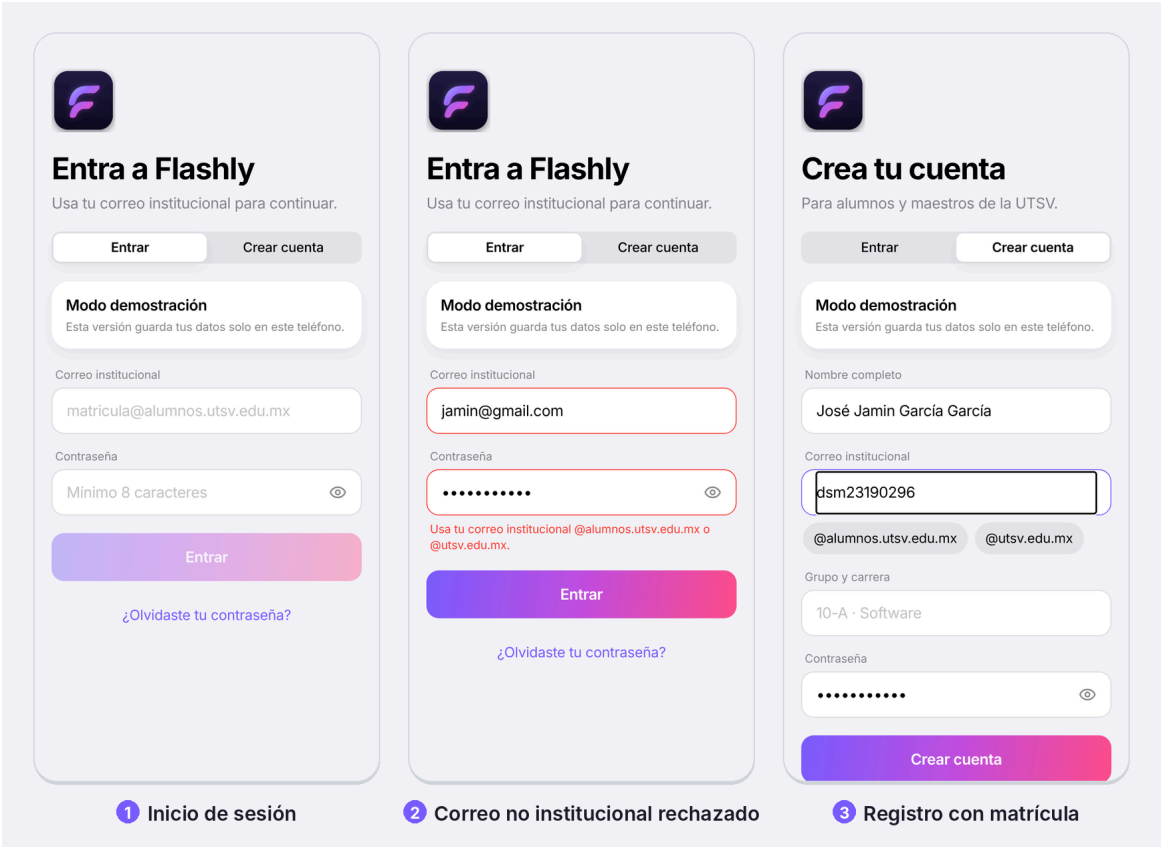



**Figura 6.** En la app: inicio de sesión, rechazo de un correo de Gmail y registro con el dominio institucional.

## 07 APIs de terceros y servicios en la nube

La API de terceros principal es la Upload API de Cloudinary: el adaptador `CloudinaryPhotoStorage` sube cada foto por HTTPS con un preset y guarda en Firestore la `secure_url` que regresa. En el servidor, la Cloud Function de moderación usa Google Cloud Vision.

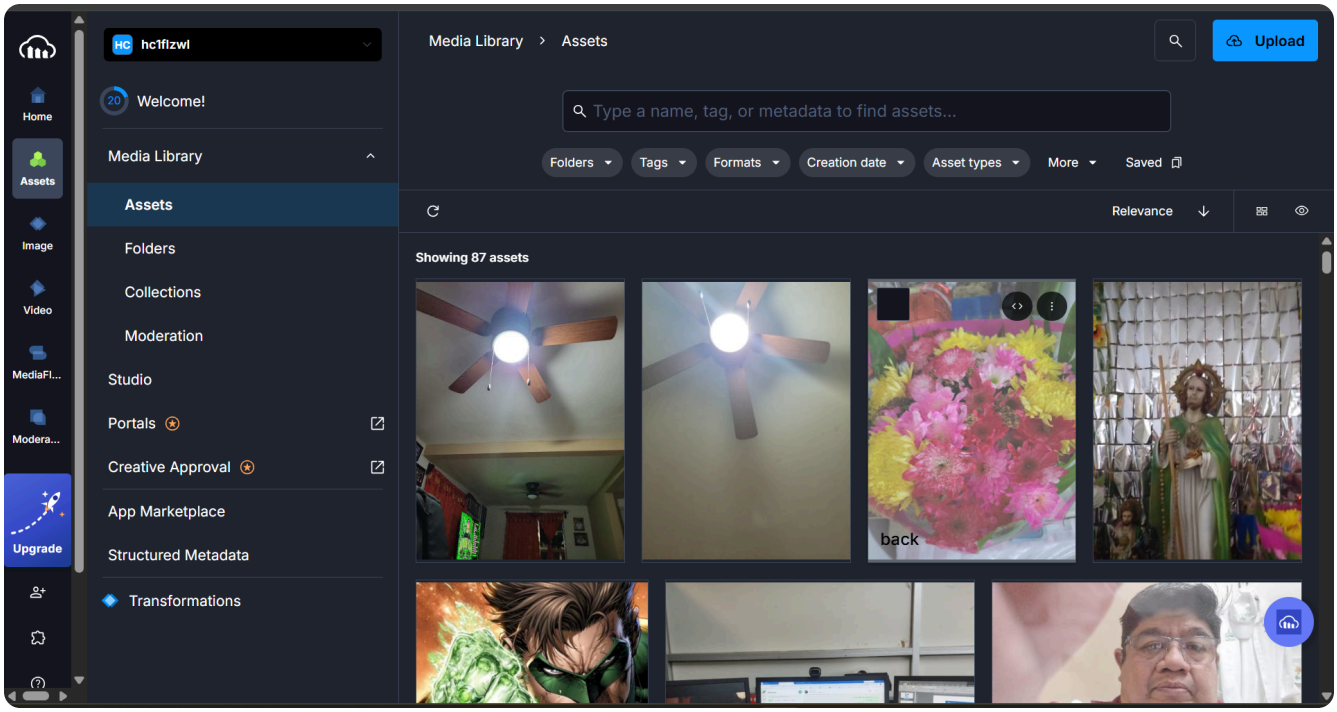



**Figura 7.** Media Library de Cloudinary ( `hc1flzwl` ) con 87 archivos subidos desde la app.

**Servicios en la nube.** El backend es el proyecto de Firebase `flashly-d0034` en el plan Spark: Authentication para las cuentas y Cloud Firestore como base de datos en tiempo real. Las Cloud Functions (ventana diaria y moderación) están listas y se despliegan con el plan Blaze; el APK se compila en Expo EAS.

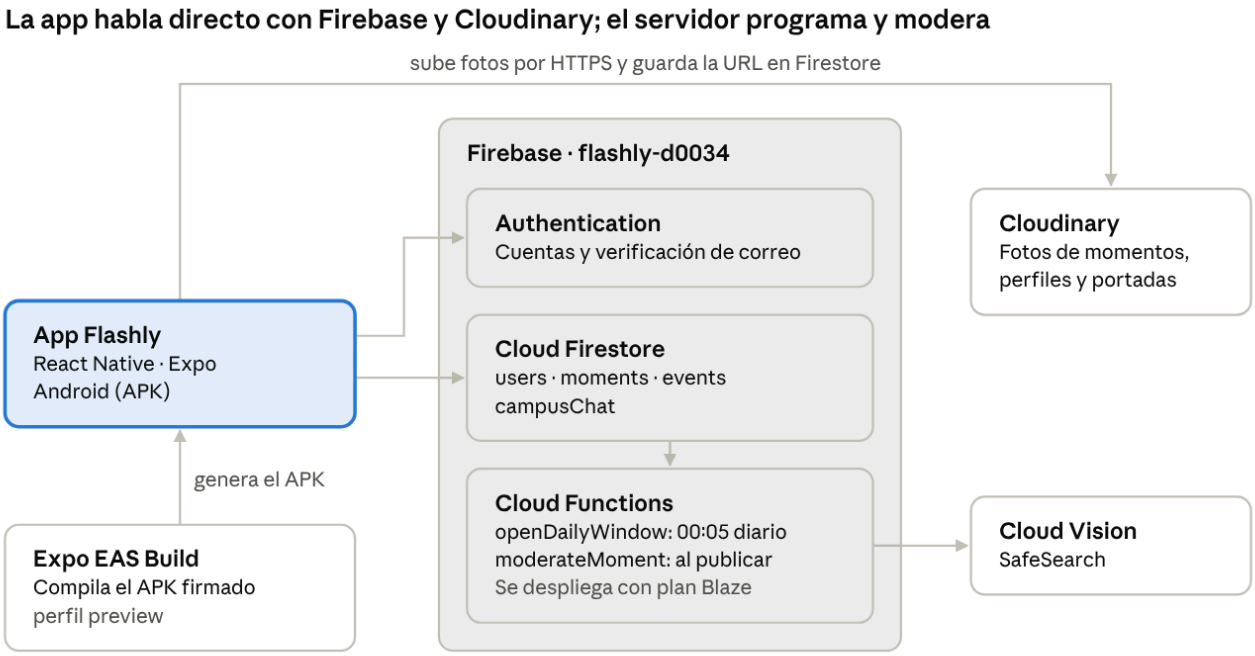



**Figura 8.** Cómo se conecta la app con los servicios en la nube.

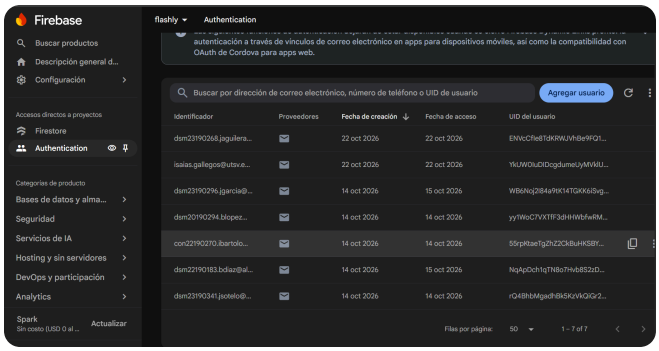


**Figura 9.** Firebase Authentication: 7 cuentas institucionales del equipo y del docente.

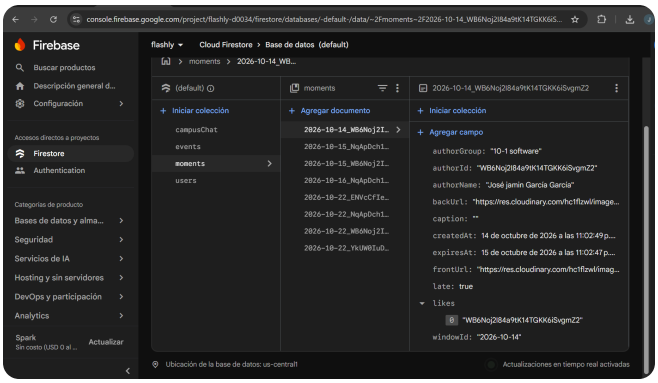


**Figura 10.** Momento real en Firestore: id por día, fotos en Cloudinary, `expiresAt` a 24 h y `late: true`.

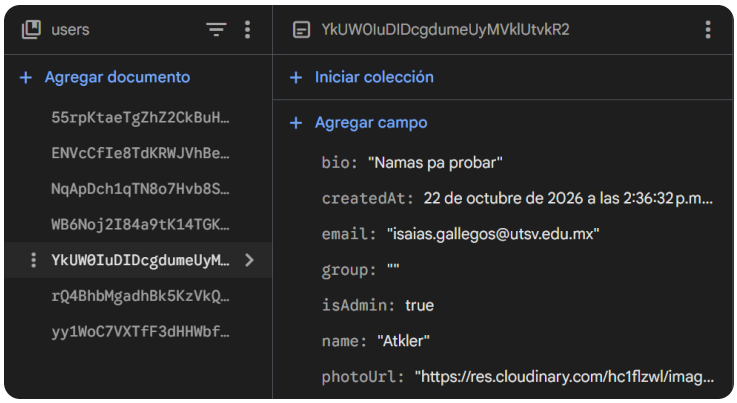


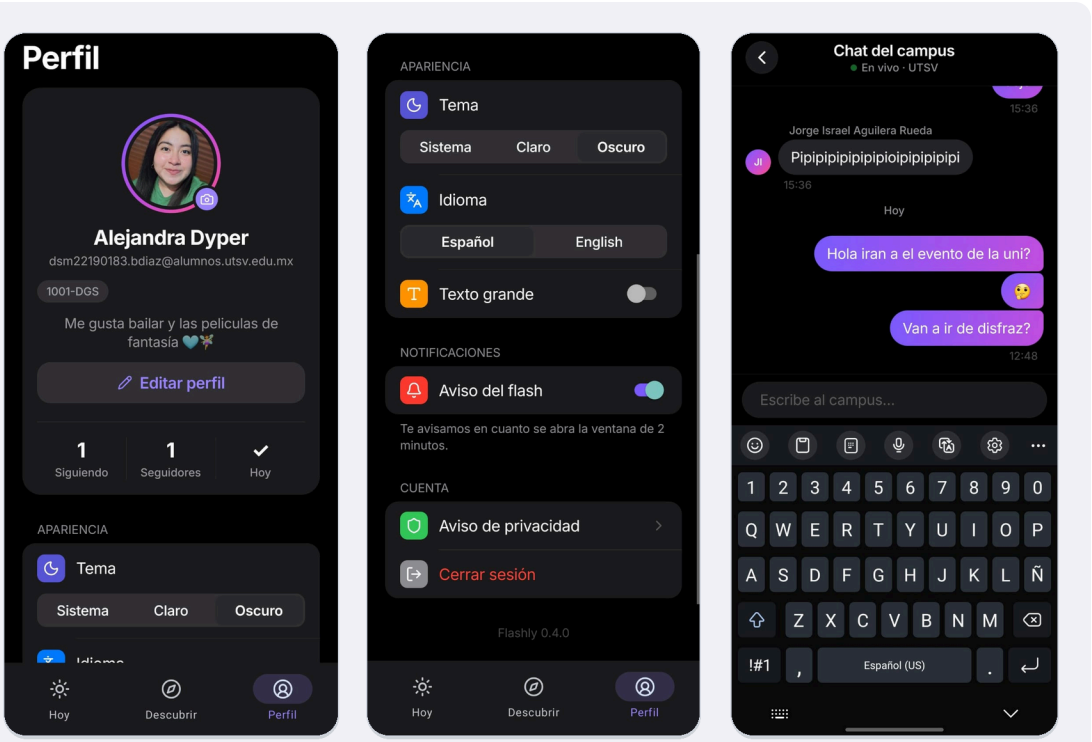

**Figura 11.** Perfil del docente con `isAdmin: true`, asignado desde la consola.

## 08 Aplicación en funcionamiento

Flashly corre en Android con el APK de la v0.4.0 y datos reales de Firestore (cuenta de Bárbara Alejandra Díaz Pérez): feed bloqueado antes del flash, momento publicado con doble cámara, lugares, perfil, ajustes y chat del campus en vivo.




**Figura 12.** APK en Android: feed bloqueado, momento publicado y Descubrir con la Cancha.




1 Perfil real en la nube

2 Ajustes: tema, idioma, aviso

3 Chat del campus en vivo

**Figura 13.** Perfil real, ajustes (tema, idioma y aviso del flash) y chat del campus con mensajes del equipo.

## 09 Enlace del repositorio y conclusiones

REPOSITORIO

**https://github.com/Jamgargar04/flashly**

v0.4.0 en la rama develop · ejecuciones de CI

El repositorio funciona desde cero: el 28 de octubre de 2026 se clonó, se instaló con `npm ci` y pasó tipos y pruebas. El APK está compilado en Expo EAS y se instala desde el enlace o el código QR:

```
expo.dev/accounts/jamingarciadev/projects/flashly/builds/7fc55cc5-2446-4d05-aba1-2152da852200
```

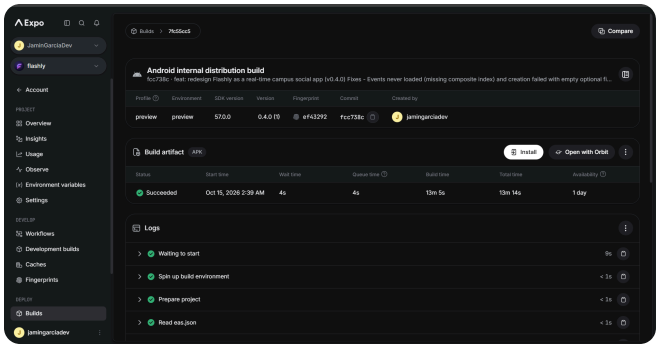



**Figura 14.** Build de EAS del commit `fcc738c` : perfil preview, versión 0.4.0 (1), terminado con éxito.

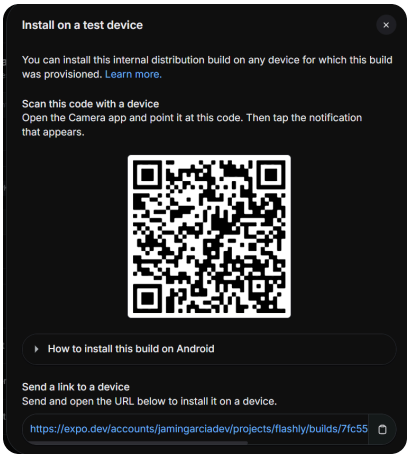



**Figura 15.** Código QR para instalar el APK en Android.

**Conclusiones.** Flashly v0.4.0 cumple el resultado de aprendizaje: es una app móvil funcional, basada en un caso real de la UTSV, versionada en GitHub con integración continua y probada con 56 pruebas automáticas. Clean Architecture y sus patrones permitieron conectar Firebase y Cloudinary sin tocar las reglas del negocio, y las reglas del servidor protegen los datos aunque se modifique la app. Los siguientes pasos son subidas firmadas a Cloudinary, pruebas de reglas en el emulador y notificaciones push desde el servidor.